

Project 2: Neighborhood-based / Model-Based Recommendation

Due: 2026/10/22, 11:59 PM

Goal

- Neighborhood-Based CF 알고리즘과 Model-Based CF 알고리즘 구현

Project Environment

- NumPy, Pandas 외 비슷한 종류의 다른 라이브러리 사용 가능, 사용하지 않고 구현해도 무방
 - o Python 3.12
 - o NumPy (optional)
 - o Pandas (optional)

Dataset

- 영화 평점 dataset
 - o ratings_train.csv | 영화에 대한 사용자의 평점 (학습용 데이터)
 - o ratings_val.csv | 영화에 대한 사용자의 평점 (학생 평가용 데이터)
 - o ratings_test.csv | 영화에 대한 사용자의 평점 (TA 채점용 데이터, **제공되지 않음**)

Task

1. User-based Collaborative Filtering
 - a. Rating 점수를 이용하여 user-item sparse matrix 생성.
 - i. "ratings_train.csv"에서 모든 데이터를 [# of unique users, # of unique movies]인 matrix로 변환
 - b. 두 user 간의 Pearson Correlation Coefficient 계산
 - i. User-item sparse matrix를 이용하여 유저들 사이의 유사도 계산

- 두 유저가 공통으로 평가한 아이템($I_{u,v}$)만 사용하여 계산

$$\text{수식: } s(u, v) = \frac{\sum_{i \in I_{u,v}} (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{\sqrt{\sum_{i \in I_{u,v}} (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_{i \in I_{u,v}} (r_{vi} - \bar{r}_v)^2}}$$

- 두 변수 간의 선형 상관관계 측정 $\Rightarrow \text{range} = [-1, 1]$
- 유저의 평균 평점($\bar{r}_u, \bar{r}_v$) 기반 보정을 통해 사용자의 개인 평가 성향 (즉, 다른 사용자에게 비해 평점을 줄 때 관대/엄격한지) 반영
- $i \in I_{u,v}$: 사용자 u 와 v 가 공통으로 평가한 아이템들의 집합
- 공통으로 평가한 아이템이 없을 시, 두 사용자 간의 유사도 = 0
- 모든 사용자와의 유사도가 0일 경우, 추천하고자 하는 아이템의 평균 평점을 예측 평점으로 취함

c. 추천 점수 계산

i. "ratings_train.csv"에서 특정 user_id가 평가한 movie_id와 rating 추출

ii. User-user similarity matrix와 rating을 이용해 해당 유저 u 가 평가하지 않은 아이템 i 에 대해 남길 추정 점수 $\hat{r}_{ui}$ 계산

$$\text{수식: } \hat{r}_{ui} = \bar{r}_u + \frac{\sum_{v \in U, v \neq u} s(u, v) \cdot (r_{vi} - \bar{r}_v)}{\sum_{v \in U, v \neq u} |s(u, v)|}$$

- $v \in U, v \neq u$: 추천을 하고자 하는 유저 u 를 제외한 사용자들 중 아이템 i 를 평가한 유사 사용자들의 집합
- r_{vi} : 다른 사용자 v 가 아이템 i 에 대해 부여한 실제 평점
- $\bar{r}_u, \bar{r}_v$: 사용자 u 와 v 의 평균 평점
- $s(u, v)$: 1.b.에서 구한 사용자 u 와 v 간의 유사도
- $|s(u, v)|$: 사용자 유사도의 절댓값 (가중치 계산 시 유사도가 음수일 수 있으므로 절대값 사용)

- 검증: prediction scores와 실제 해당 movie의 평가 점수들 간 RMSE
 - a. user_id=199 기준 0.65미만
 - b. user_id=458 기준 0.75미만
 - c. user_id=529 기준 0.75미만

d. 계산한 추천 점수와 "ratings_val.csv"의 RMSE 계산

e. "ratings_val.csv"에 있고 "ratings_train.csv"에 없는 movie는 계산에서 제외

i. Cold-start items는 추천 대상에서 제외

2. Matrix Factorization

- a. Rating 점수를 이용하여 user-item sparse matrix 생성
 - i. "ratings_train.csv"에서 모든 데이터를 [# of unique users, # of unique movies]인 matrix로 변환
- b. User-item sparse matrix의 빈자리 채우기
 - i. Rating이 0으로 되어 있으면 0에 근접하게 학습되기에 각 영화의 평균 평점으로 대체
- c. Decomposition of the rating matrix
 - i. numpy.linalg.svd를 이용하여 user factors와 item factors를 생성
 - ii. Singular value 중 가장 큰 400개의 요소만 사용 ($K = 400$)
- d. 추천 점수 계산
 - i. User factors와 item factors를 matmul하여 추정 점수 계산
- e. 계산한 추천점수와 "ratings_val.csv"의 RMSE 계산
 - i. "ratings_val.csv"에 있고 "ratings_train.csv"에 없는 movie와 user는 계산에서 제외
 - ii. 검증: prediction scores와 실제 해당 movie의 평가 점수들 간 RMSE
 - user_id=199 기준 0.6미만
 - user_id=458 기준 0.7미만
 - user_id=529 기준 0.75미만

3. Optimization

- a. Task 1 또는 Task 2의 방법 기반으로 공식 혹은 parameters를 변경해보며 추천 성능 향상
- b. 추천 점수 계산 시 "ratings_train.csv" 데이터만 사용하여야 함. "ratings_val.csv"는 성능 평가 시에만 사용

평가기준

1. 특정 user_id, movie_id에 대한 prediction_score로 평가
 - a. Task 1: User-based collaborative filtering 방법으로 계산한 점수와의 허용 범위 내 오차
 - b. Task 2: Matrix factorization 방법으로 계산한 점수와의 허용 범위 내 오차
 - c. Task 3: 별도의 optimization 방법으로 계산한 prediction_score와 실제 rating 사이의 RMSE
2. 포맷 준수 여부
 - a. "input.txt"의 user_id, movie_id마다 해당하는 점수를 3줄씩 출력.
(첫째 줄: User-based, 둘째 줄: Matrix factorization, 셋째 줄: Optimized result)
 - b. "output.txt"의 필드 (user_id, movie_id, prediction_score)간 구분자는 공백 없이 콤마(,)를 사용
 - c. prediction_score의 경우, 소수점 4번째까지 반올림
 - d. "input.txt", "output.txt" 파일은 "main.py" 파일과 같은 위치에 저장

작성한 코드를 조교가 ratings_train.csv로 학습 후 training이 잘 이루어졌는지 확인할 예정 (ratings_val.csv는 학습 확인 용도로 학습 시 사용하지 말 것)

Example

```
# input.txt format
# 한 줄에 user_id, movie_id
1,31
2,10
3,1235
4,10
...
```

```
# output.txt format
# 순서대로 (user_id, movie_id, prediction_score) 출력
# 아래 출력 결과는 출력 형태 예시를 보일 뿐, 실제 정답과는 무관함
1,31,2.5000
1,31,2.5000
1,31,2.5000
2,10,4.0000
2,10,4.0000
```

```
2,10,4.0000
3,1235,4.0000
3,1235,4.0000
3,1235,4.0000
...
```

Source Code

- main.py를 기반으로 구현

Report

- 다음과 같은 내용을 보고서에 간단하게 작성
 - Task 1, 2의 간단 설명
 - Task 3에 대한 상세설명
 - 최적화 Task 대상 (Task1 or Task2)
 - 주요 최적화 로직 설명 및 근거
 - 모델 개선 결과 분석
 - 본 프로젝트에 대한 피드백 의견
- 길이는 최대 A4 4장까지, PDF 형식으로 제출
- 파일 제목은 다음과 같은 양식을 준수: PRJ2_202600000_홍길동.pdf

Submission

- ETL 과제 탭으로 제출
- Source code와 Report를 함께 zip 파일로 묶어서 첨부
- 제목 양식은 다음을 준수: PRJ2_202600000_홍길동.zip
- 지연 제출
 - 제출 기한 이후 24시간 이내 제출시 10% 감점
 - 제출 기한 이후 24시간 이후, 48시간 이내 제출시 20% 감점
 - 제출 기한 이후 48시간 이후에는 점수 없음

- 부정 행위는 0점 처리
 - 다른 사람의 코드를 참조하는 행위
 - 이전에 수강한 사람의 코드를 참조하는 행위
 - 표절 방지 프로그램을 통해 표절 여부 확인할 예정
- 본 문서에 명시되어 있는 제출 / 출력 양식을 지키지 않을 시 감점

제출 상태

PRJ2_202600000_홍길동.zip

```
|— PRJ2_202600000_홍길동.pdf
|— data
|   |— ratings_train.csv
|   └─ ratings_val.csv
|— main.py
|— input.txt (제출 x, 채점 시 위치)
└─ output.txt (제출 x, 채점 시 생성)
```